Assignment 3: Constant-Time AES Implementation

Problem Statement

AES-128 [1] is one of the most widely used symmetric cryptographic primitives. It takes a **128-bit plaintext** and a **128-bit master key** as input and produces a **128-bit ciphertext**.

AES operates in **10 rounds**, where each round takes:

- The output of the previous round (plaintext for the first round)
- A **round key** derived from the master key

Before the first round:

$$\text{State} = \text{Plaintext} \oplus \text{Master Key}$$

Each round input is represented as:

$$\text{State} = \begin{bmatrix} s_{0,0} & s_{0,1} & s_{0,2} & s_{0,3} \\ s_{1,0} & s_{1,1} & s_{1,2} & s_{1,3} \\ s_{2,0} & s_{2,1} & s_{2,2} & s_{2,3} \\ s_{3,0} & s_{3,1} & s_{3,2} & s_{3,3} \end{bmatrix}$$

Each round consists of:

1. SubBytes
2. ShiftRows
3. MixColumns
4. AddRoundKey

Note: The final round does not include MixColumns.

Description of Operations

1. SubBytes

$$s_{i,j} = A(\text{inv}(s_{i,j})) \oplus 0x63$$

where:

$A(x) = x \oplus (x \lll 1) \oplus (x \lll 2) \oplus (x \lll 3) \oplus (x \lll 4)$; $\lll$ denotes left bitwise circular shift.

$inv(x)$ is multiplicative inverse in $GF(2^8)$ over irreducible polynomial $x^8 + x^4 + x^3 + x + 1$

2. ShiftRows

$$\begin{bmatrix} s_{0,0} & s_{0,1} & s_{0,2} & s_{0,3} \\ s_{1,0} & s_{1,1} & s_{1,2} & s_{1,3} \\ s_{2,0} & s_{2,1} & s_{2,2} & s_{2,3} \\ s_{3,0} & s_{3,1} & s_{3,2} & s_{3,3} \end{bmatrix} \rightarrow \begin{bmatrix} s_{0,0} & s_{0,1} & s_{0,2} & s_{0,3} \\ s_{1,1} & s_{1,2} & s_{1,3} & s_{1,0} \\ s_{2,2} & s_{2,3} & s_{2,0} & s_{2,1} \\ s_{3,3} & s_{3,0} & s_{3,1} & s_{3,2} \end{bmatrix}$$

3. MixColumns

$$\begin{bmatrix} s_{0,j} \\ s_{1,j} \\ s_{2,j} \\ s_{3,j} \end{bmatrix} \rightarrow \begin{bmatrix} 2 & 3 & 1 & 1 \\ 1 & 2 & 3 & 1 \\ 1 & 1 & 2 & 3 \\ 3 & 1 & 1 & 2 \end{bmatrix} \cdot \begin{bmatrix} s_{0,j} \\ s_{1,j} \\ s_{2,j} \\ s_{3,j} \end{bmatrix}$$

All operations are in GF(2^8). This operation is not present in the final round.

4. AddRoundKey

$$s_{i,j} \rightarrow s_{i,j} \oplus k_{i,j}^{(\text{round})}$$

Security Analysis

In optimized implementations, SubBytes is implemented using a lookup table:

SBOX[a]

where:

$$a = s_{i,j} \oplus k_{i,j}^{(\text{round})}$$

This results in a memory access:

$$\text{address} = \text{base} + (s_{i,j} \oplus k_{i,j}^{(\text{round})})$$

Security Implication

In the first round, If an attacker observes memory accesses and knows $p_{i,j}$:

$$k_{i,j}^{(0)} = (\text{address} - \text{base}) \oplus p_{i,j}$$

Thus, the secret key can be recovered as it results in a secret dependent memory access based on $k_{i,j}^{(\text{round}))}$.

T-Table Optimization

High-performance implementations use T-tables, which also result in memory access indexed by $plaintext \oplus key$

Objective

Implement AES-128 encryption such that there are **no secret-dependent memory accesses**.

Files Provided

File	Description
AES_code.c	AES implementation file, where you have to implement your AES.
analysis.c	Memory trace generator
check.py	Leakage testing script

In AES_code.c we have provided an OpenSSL-based AES encryption implementation for your reference. Your task is to implement AES-128 encryption only. Key expansion is already handled by the OpenSSL AES module. The round key is passed to your implementation in the following structure AES_KEY. The structure definition 'AES_KEY_Custom' is already available in 'AES_code.c'. You must directly use the provided round keys for encryption.

Important Instructions

- Modify only `AES_code.c`
- Do not modify other files
- Implement encryption only
- Use provided round keys

Forbidden Implementations

- Lookup-table S-box
- Secret-dependent memory access
- AES-NI or ISA-specific instructions

Testing

```
python3 check.py
```

The script will:

1. Generate two random 128-bit plaintexts
2. Execute your AES implementation
3. Record memory accesses during encryption
4. Compare access patterns

Output:

- Pass: Test Passed
- Fail: Test Failed

Performance Evaluation

- Use plaintext: `0xffffffffffffffffffffffffffffffff`
- Measure different type of instruction count like you did in assignment 1.
- Compute average CPI like you did in assignment 1.
- Compare CPI of openSSL with your implementation.

Running the Open-SSL test on this implementation will result in test failed. This is expected because OpenSSL uses lookup tables that introduce secret-dependent memory accesses.

Grading Policy

The group with the best CPI will receive 100 marks. This CPI will be treated as the reference CPI. Your marks will be assigned based on relative CPI.

Relative CPI	Marks
Within 10%	95
Within 20%	85
Within 30%	75
Else	60

Submission Checklist

- Modified AES_code.c
- CPI calculation
- Performance report

Reference

[1] https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.197-upd1.pdf